

1.Graph란 무엇인가

Agentic Workflow란 무엇인가

Agentic Workflow는 Agent가 사용자의 요청을 받고, 필요한 정보를 판단한 뒤, 도구를 사용하고, 결과를 정리하는 업무 처리 흐름입니다.

Graph는 이 Agentic Workflow를 표현하기 위한 구조입니다.



Graph는 Agentic Workflow를 표현하는 구조입니다.
즉, Agent가 어떤 순서로 판단하고 행동할지 정하는 실행 흐름입니다.

📌 핵심 정리: Agentic Workflow는 Agent가 판단하고, 도구를 사용하고, 결과를 정리하는 업무 처리 흐름입니다.



uv 실행 명령어 전체 정리

이 데크에서 사용하는 모든 uv 실행 명령어를 한 곳에 정리합니다. 프로젝트 루트 폴더에서 실행합니다.

mini_graph_runner.py

```
uv run python src/day1/mini_graph_runner.py
```

day1_agent_v0_template.py

```
uv run python  
src/day1/day1_agent_v0_template.py
```

Streamlit

```
uv run streamlit run src/day1/mini_graph_streamlit_app.py
```

```
uv run streamlit run src/day1/day1_agent_v0_streamlit_app.py
```



모든 명령어는 프로젝트 루트 폴더(PowerShell)에서 실행합니다.

Graph란 무엇인가

Graph는 조건에 따라 실행 흐름이 달라지는 구조입니다. Chain은 정해진 순서대로만 진행되지만, Graph는 상황을 보고 다음 길을 선택합니다.

내비게이션 비유

내비게이션이 길 상황에 따라 경로를 바꾸는 것처럼, Graph도 질문 상태에 따라 다음 작업을 다르게 선택합니다.

제조 장애 대응 예시

- 사용자 질문에 설비 ID와 알람 코드가 있는지 먼저 확인합니다.
- 정보가 충분하면 → 로그 조회로 이동합니다.
- 정보가 부족하면 → 추가 질문을 요청합니다.



핵심 정리: Graph는 Agent가 상황에 맞게 길을 고르는 기본 흐름입니다.

Chain과 Graph의 차이

Chain과 Graph는 실행 방식에서 근본적인 차이가 있습니다. 아래 표를 통해 두 구조를 비교해 보겠습니다.

구분	Chain	Graph
실행 방식	항상 같은 순서로 실행	조건에 따라 흐름 변경
흐름 특징	직선형 흐름	갈림길이 있는 흐름
제조 장애 대응 예시	질문 → Prompt 생성 → LLM 호출	알람 코드 있음/없음에 따라 다른 처리
적합한 상황	절차가 고정된 작업	입력 상태에 따라 판단이 필요한 작업

Chain = 요리 순서

요리 레시피처럼 정해진 단계를 순서대로 따릅니다. 항상 같은 길을 걷습니다.

Graph = 내비게이션

도로 상황에 따라 다음 경로를 고릅니다. 조건에 따라 길이 달라집니다.

✅  핵심 정리: Graph는 실제 업무처럼 상황에 따라 달라지는 흐름을 표현하기 좋습니다.

알람 코드 있음/없음 조건 분기

제조 장애 대응에서 알람 코드가 있는지 없는지는 매우 중요한 판단 기준입니다.

✅ 알람 코드가 있는 경우

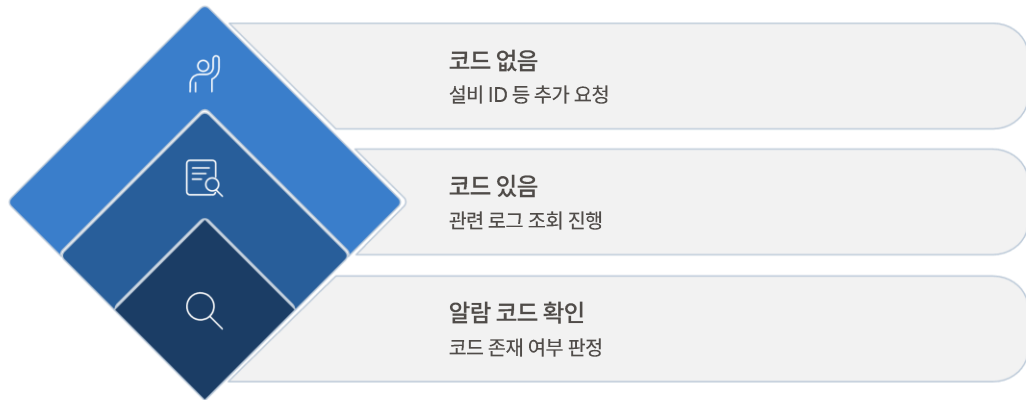
Agent는 관련 로그를 찾고 매뉴얼 근거를 확인할 수 있습니다.

예시: EQP-EV-03 + ALM-TEMP-402 → 로그 조회로 이동

❌ 알람 코드가 없는 경우

"알람 원인 알려줘"라고만 말하면 설비 ID와 알람 코드가 부족합니다.

바로 답하지 않고 추가 정보를 요청해야 합니다.

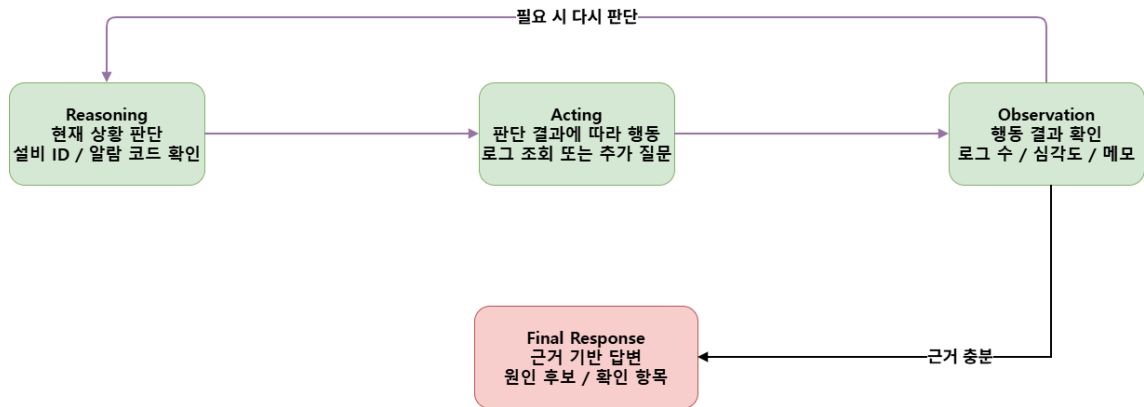


🔴 핵심 정리: Graph는 정보가 충분한 경우와 부족한 경우를 나누어 처리합니다.

Reasoning & Acting 구조

Agent는 바로 답변하지 않고 먼저 현재 상황을 판단합니다.

그리고 판단 결과에 따라 필요한 행동을 선택합니다.



★ 핵심 정리: Agent는 판단한 뒤 행동하고, 행동 결과를 확인한 뒤 최종 답변을 만듭니다.

mini_graph_runner.py 파일 역할

src/day1/mini_graph_runner.py는 조건부 Graph 흐름을 이해하기 위한 실습 파일입니다.

01

입력 확인

사용자 질문과 입력값을 확인합니다.

03

분기 실행

두 값이 있으면 search_log_node로 이동해 관련 로그를 찾습니다.

02

조건 검사

equipment_id와 alarm_code가 있는지 검사합니다.

04

추가 질문

값이 부족하면 ask_more_info_node로 이동해 추가 정보 요청 메시지를 만듭니다.

실행 명령어

```
uv run python src/day1/mini_graph_runner.py
```

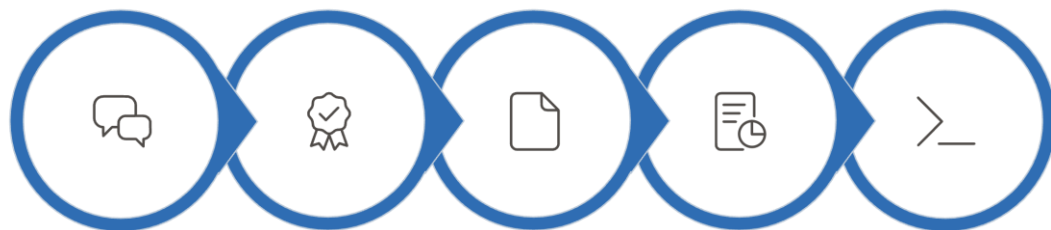
프로젝트 루트 폴더에서 실행합니다.

  이 파일은 단순 실행 순서가 아니라 조건에 따라 다음 작업이 달라지는 업무 흐름을 보여줍니다.

  핵심 정리: mini_graph_runner.py는 Agent의 갈림길 판단을 체험하는 파일입니다.

알람 코드가 있는 경우

알람 코드가 있는 경우의 흐름은 비교적 자연스럽습니다. 사용자 요청에 `equipment_id`와 `alarm_code`가 모두 있으면 Graph는 필수 정보가 있다고 판단합니다.



사용자 질문

알람 코드 확인

로그 조회

결과 요약

Prompt 생성

1단계: 알람 코드 확인

`equipment_id`와 `alarm_code`가 모두 있는지 판단합니다.

2단계: 로그 조회

`search_log_node`로 이동해 `sample_alarm_logs.csv`에서 조건에 맞는 로그를 찾습니다.

3단계: 결과 요약 및 Prompt 생성

결과를 요약하고 Prompt를 만든 뒤 LLM 응답 흐름으로 이어 집니다.



✦ 핵심 정리: 정보가 충분하면 Agent는 필요한 자료를 찾아 다음 단계로 진행합니다.

알람 코드가 없는 경우

알람 코드나 설비 ID가 없으면 Graph는 바로 답변을 만들지 않습니다.

✗ 이런 질문은 정보가 부족합니다

| "알람 원인 알려줘"

어떤 설비인지, 어떤 알람인지 알 수 없습니다. 이때 `mini_graph_runner.py`는 `ask_more_info_node`로 이동해 추가 정보가 필요하다는 메시지를 만듭니다.

Agent vs 단순 LLM 차이

단순 LLM: 부족한 정보로도 답을 만들 수 있습니다.

Agent: 필요한 정보가 부족하면 먼저 질문해야 합니다.

Agent는 무리한 추측보다 안전한 추가 질문을 선택합니다.

⚠ ▲ 정보가 부족할 때 무리하게 답변을 만들면 잘못된 정보가 전달될 수 있습니다.

✅ ✳ 핵심 정리: Graph는 무리한 추측보다 안전한 추가 질문을 선택할 수 있습니다.

조건부 흐름 정리

이번 장에서는 Graph가 조건에 따라 실행 경로를 바꾸는 구조라는 점을 배웠습니다.



정보가 있는 경우

설비 ID + 알람 코드 확인

→ 로그 조회

→ 매뉴얼 확인

→ 결과 생성



정보가 부족한 경우

설비 ID 또는 알람 코드 없음

→ 추가 질문 생성

→ 사용자에게 재요청



Mermaid 같은 별도 다이어그램을 사용하지 않아도 텍스트 흐름도와 표만으로 충분히 이해할 수 있습니다.



다음 장 예고: 이 흐름을 **Agent v0**로 연결합니다.



핵심 정리: Graph는 Agent가 질문 상태를 보고 다음 행동을 선택하게 해 주는 구조입니다.

2.Day1 Agent v0 예제와 Streamlit 실습

Agent v0란 무엇인가

Agent v0는 1일차에 배운 Prompt, Chain, Graph를 연결한 작은 제조 장애 대응 Agent입니다. 완성형 Agent는 아니지만, Agent의 기본 아이디어를 담고 있습니다.

01

사용자 질문 수신

사용자 질문을 받습니다.

02

정보 확인

필요한 정보(설비 ID, 알람 코드)를 확인합니다.

03

흐름 선택

조건에 따라 흐름을 선택합니다.

04

Prompt 생성 및 LLM 호출

Prompt를 만든 뒤 실제 LLM API 호출 흐름으로 응답을 생성합니다.



자동차 비유: Agent v0는 완성차가 아니라 기본 뼈대와 바퀴를 조립한 첫 번째 버전입니다.



핵심 정리: Agent v0는 복잡한 Agent로 가기 전 가장 작은 실습용 구조입니다.

Agent v0 구성 요소

Agent v0는 질문을 바로 답으로 바꾸는 것이 아니라 여러 단계를 거쳐 업무 결과를 만듭니다.

구성 요소	역할
사용자 질문	Agent가 처리할 첫 입력입니다.
알람 코드 확인	설비 ID와 알람 코드가 있는지 봅니다.
로그/매뉴얼 조회	관련 로그와 문서 근거를 찾습니다.
Prompt 생성	LLM에게 전달할 작업 지시문을 만듭니다.
실제 LLM API 호출	Prompt를 LLM에 보내 응답을 받습니다.
결과 출력	사용자가 볼 수 있는 결과로 정리합니다.



업무 조교 비유: Agent v0는 자료를 찾아 정리하는 업무 조교처럼, 단계별로 일을 처리한 뒤 최종 결과를 전달합니다.

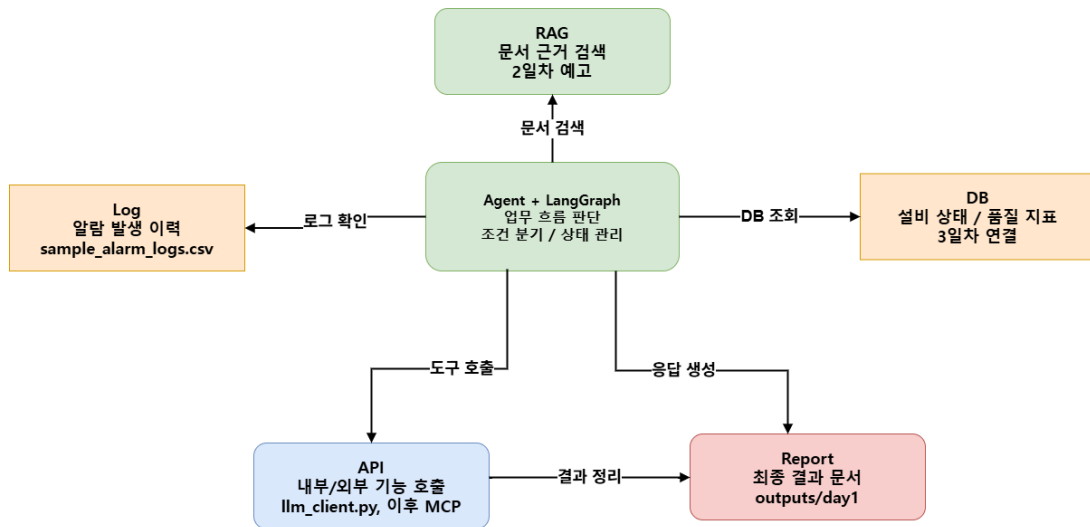


핵심 정리: Agent v0는 질문을 바로 답으로 바꾸는 것이 아니라 여러 단계를 거쳐 업무 결과를 만듭니다.

제조 업무형 Agent 구성요소 역할 정리

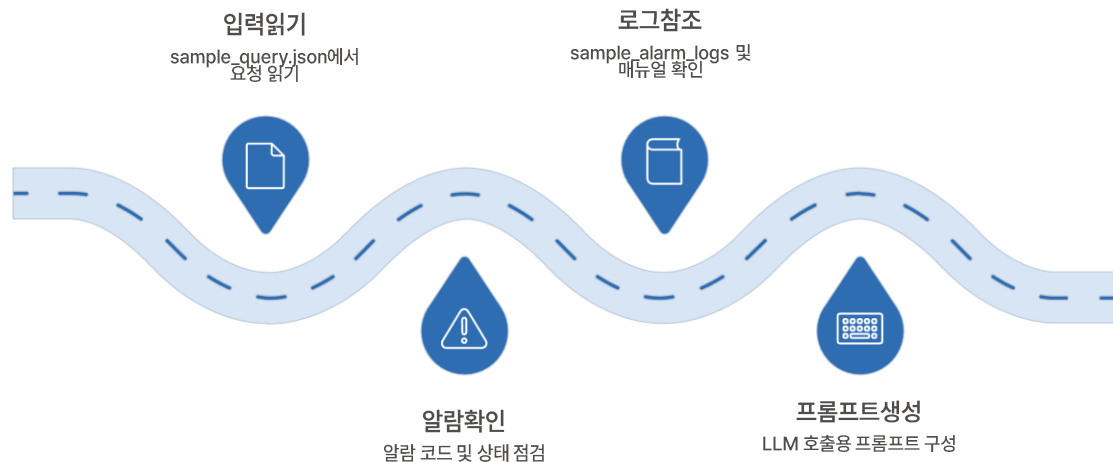
Agent v0는 1일차 실습용 작은 구조입니다.

실제 제조 업무형 Agent는 문서 검색, 로그 조회, DB 조회, API 호출, 보고서 작성까지 확장됩니다.



Agent v0 실행 흐름

Agent v0의 실행 흐름은 1일차 실습을 하나로 묶은 형태입니다.



입력 파일

sample_query.json에서 사용자 요청을 읽습니다.

LLM 호출

llm_client.py의 generate_response(prompt)를 통해 LLM 응답을 받습니다.

조회 파일

sample_alarm_logs.csv와 alarm_manual.md를 참고합니다.

결과 저장

결과와 Trace를 outputs/day1에 저장합니다.



✦ 핵심 정리: Agent v0는 입력, 판단, 조회, Prompt, LLM 응답, 저장을 연결한 흐름입니다.

day1_agent_v0_template.py 선택 실행

src/day1/day1_agent_v0_template.py는 Agent v0 전체 흐름을 보여주는 파일입니다.

권장 학습 순서

- 1 first_sample_run.py**
가장 작은 흐름을 먼저 이해합니다.
- 2 simple_chain_starter.py**
Chain 흐름을 이해합니다.
- 3 mini_graph_runner.py**
조건 분기 흐름을 이해합니다.
- 4 day1_agent_v0_template.py**
전체 구조를 한눈에 확인합니다.

실행 명령어

```
uv run python src/day1/day1_agent_v0_template.py
```

프로젝트 루트 폴더에서 실행합니다.



🌟 핵심 정리: 이 파일은 1일차 조각 실습을 하나로 연결해 보는 종합 예시입니다.

실행 결과 해석

day1_agent_v0_template.py를 실행하면 Agent v0가 어떤 단계를 거쳤는지 확인할 수 있습니다. 결과 파일은 outputs/day1 폴더에 저장될 수 있습니다.

day1_agent_v0_result.md

최종 결과를 보여줍니다.

day1_agent_v0_trace.md

어떤 Node를 거쳤는지 보여줍니다.

결과를 볼 때 확인해야 할 항목

- 입력값이 무엇이었는지 확인합니다.
- 로그 조치가 이루어졌는지 확인합니다.
- 매뉴얼 참고 여부를 확인합니다.
- Prompt 생성 여부를 확인합니다.
- 조건 분기 경로가 어떻게 결정되었는지 확인합니다.



▲ LLM 응답만 보지 말고 입력값과 실행 과정을 함께 확인해야 합니다.



✳ 핵심 정리: Agent 결과는 최종 답변과 실행 과정을 함께 봐야 제대로 이해할 수 있습니다.

Streamlit 실습 개요

Streamlit은 Python 실행 결과를 웹 화면으로 보여주는 도구입니다. 1일차에서는 CLI 실행 흐름을 눈으로 확인하기 위한 보조 화면으로 사용합니다.

Streamlit의 역할

Streamlit이 Agent의 핵심 로직을 새로 만드는 것은 아닙니다.
기존 CLI 실습 흐름을 수강생이 더 쉽게 볼 수 있도록 화면으로 보여주는 역할입니다.

보조 실습 파일 목록

- `mini_graph_streamlit_app.py`
Graph 흐름을 화면에서 확인합니다.
- `day1_agent_v0_streamlit_app.py`
Agent v0 흐름을 화면에서 확인합니다.

  Streamlit은 이해를 돕는 보조 화면입니다. Agent의 핵심 로직은 CLI 실습 파일에 있습니다.

  핵심 정리: Streamlit은 이해를 돕는 보조 화면입니다.

Streamlit 실행 명령어

Streamlit 보조 실습은 선택적으로 실행합니다. PowerShell에서 프로젝트 루트 폴더로 이동한 뒤 실행합니다.

  Graph 흐름 화면 확인

```
uv run streamlit run src/day1/mini_graph_streamlit_app.py
```

  Agent v0 흐름 화면 확인

```
uv run streamlit run src/day1/day1_agent_v0_streamlit_app.py
```

추가로 사용할 수 있는 파일

simple_chain_streamlit_app.py

Chain 흐름 화면 확인

first_sample_run_streamlit_app.py

첫 번째 실행 흐름 화면 확인



▲ Streamlit은 핵심 학습 목표가 아니라 CLI 흐름을 보기 쉽게 확인하는 도구입니다.



✳️ 핵심 정리: Streamlit은 실습 흐름을 화면으로 확인하는 보조 수단입니다.

CLI와 Streamlit 결과 비교

두 가지 실행 방식을 비교하고, 상황에 맞게 선택해서 사용할 수 있습니다.

구분	CLI	Streamlit	수강생 관점의 장점
실행 방식	PowerShell에서 명령어 실행	웹 화면에서 입력 후 실행	두 방식 모두 흐름 이해에 도움
결과 확인	글자 중심으로 확인	입력값과 결과를 화면에서 확인	초보자는 화면이 더 직관적
장점	실제 개발 실행 방식 이해	실행 흐름을 눈으로 보기 쉬움	상황에 맞게 선택 가능

다음 단계 안내

1일차에서 실행한 파일과 결과를 산출물로 정리합니다.

2일차 예고: RAG 학습

Agent v0의 한계를 바탕으로, 2일차에는 Agent가 직접 문서를 검색해 근거를 찾는 RAG(검색 증강 생성) 학습으로 넘어갑니다.

🟢  1일차 핵심 정리: Prompt → Chain → Graph → Agent v0 흐름을 통해 Agent의 기본 구조를 이해했습니다. 수고하셨습니다! 🎉

3.산출물 정리와 2일차 RAG 연결

오늘 만든 산출물 목록

산출물 정리는 오늘 만든 결과를 책상 위에 정리해 두는 것과 비슷합니다. 정리하면, 산출물은 1일차 실습이 실제로 어떤 흐름으로 진행되었는지 확인하는 근거입니다.

산출물	확인 내용	의미
first_sample_run.py 실행 흐름	알람 로그와 매뉴얼 확인	Agent가 사용할 재료 확인
simple_chain_starter.py 실행 흐름	Prompt → 실제 LLM 응답 흐름 확인	정해진 순서의 업무 흐름 이해
mini_graph_runner.py 실행 흐름	알람 코드 있음/없음 분기 확인	조건부 흐름 이해
day1_agent_v0_template.py 선택 실행	Agent v0 결과와 Trace 확인	1일차 조각 실습 통합



핵심 정리: 각 산출물은 Agent의 기본 아이디어를 단계별로 이해하고 검증하는 중요한 근거입니다.

Agent v0의 한계와 의미

Agent v0는 1일차 실습의 통합 버전이지만, 완전한 형태의 Agent는 아닙니다. 오히려 다음 단계로 확장하기 위한 중요한 출발점입니다. 현재 Agent v0의 주요 한계점은 다음과 같습니다.

- 여러 문서 중 관련 근거를 자동으로 깊게 검색하지 못합니다.
- 복잡한 DB Tool이나 MCP Tool과 같은 외부 도구와 연결되지 않았습니다.
- Trace나 Guardrail 기반의 품질 평가 기능이 제한적입니다.



자동차 비유: Agent v0는 완성차가 아니라 기본 뼈대와 바퀴를 붙인 첫 번째 버전입니다.

즉, Agent v0는 사용자가 제공한 정보와 단순 조회 결과를 중심으로 동작합니다. 이는 2일차부터 5일차까지 추가될 기능을 위한 기본 구조라고 이해할 수 있습니다.



핵심 정리: Agent v0는 실패한 구조가 아니라, Agent의 핵심 기능을 차례로 붙여나가기 위한 견고한 토대입니다.

사람이 직접 매뉴얼 문장을 찾는 방식의 한계

1일차 실습에서는 사용자가 알람 로그와 매뉴얼 문장을 직접 확인하고, 필요한 문장을 골라 Prompt에 넣어 LLM 응답이 어떻게 바뀌는지 확인했습니다. 이 방식은 문서가 적을 때는 가능합니다.

하지만 매뉴얼이 길어지고 문서가 많아지면 사람이 매번 관련 문장을 찾는 것은 어렵고, 중요한 근거를 놓칠 수도 있습니다. 이는 마치 큰 도서관에서 책 제목도 모른 채 직접 책장을 뒤지는 것과 같습니다.

따라서 문서가 많아질수록 Agent가 직접 문서를 검색하여 필요한 근거를 찾는 구조가 필요합니다.

  이러한 Agent v0의 한계가 바로 2일차부터 학습할 RAG(검색 증강 생성) 기술의 필요성을 보여줍니다.

  핵심 정리: 방대한 정보 속에서 Agent가 스스로 관련 근거를 찾아내는 능력이 필수적입니다.

2일차: RAG 구조로 연결

RAG는 Retrieval-Augmented Generation의 줄임말입니다. 초보자에게는 "문서를 먼저 검색한 뒤, 찾은 근거를 LLM에게 전달해서 답변을 만드는 방식"으로 이해하면 됩니다. 1일차에는 사람이 매뉴얼 문장을 찾아 Prompt에 넣었지만, 2일차에는 Agent가 관련 문서를 직접 찾도록 확장합니다.



정리하면, RAG는 Agent가 문서 창고에서 필요한 근거를 찾아오는 방식입니다.



🔴 핵심 정리: RAG는 Agent가 방대한 문서에서 필요한 정보를 스스로 찾아 LLM 응답을 강화하는 기술입니다.

2일차 학습 예고: RAG Agent로의 전환

1일차 Agent v0는 2일차 RAG Agent로 확장되며, Agent가 스스로 문서를 검색하여 근거를 찾아내는 능력을 갖추게 됩니다.

1

문서 처리

Markdown 문서를 Python으로 읽고, 긴 문서를 작은 조각인 **Chunk**로 나눕니다.

2

메타데이터 부착

각 Chunk에 **Metadata**(이름표)를 부착하여 정보 관리를 용이하게 합니다.

3

문서 조각 검색

질문과 관련 있는 **Top-3** 문서 조각을 효과적으로 찾아냅니다.

4

State 저장

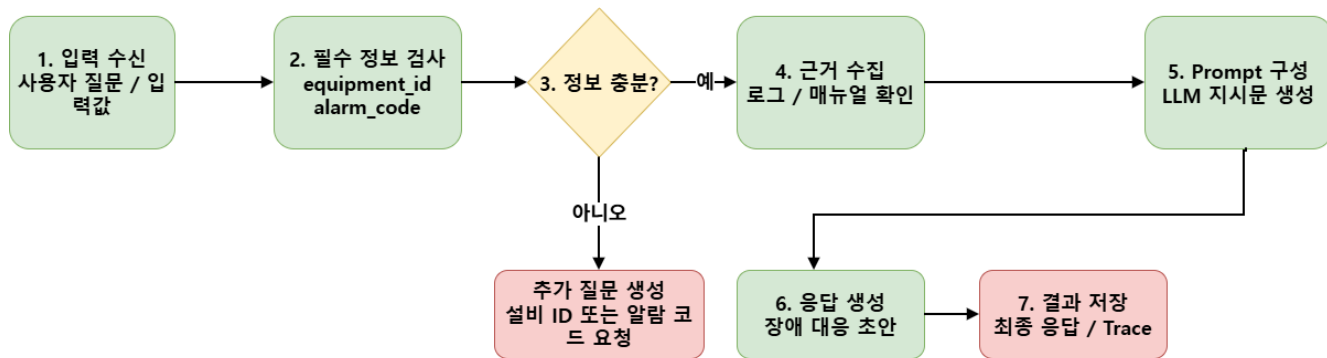
검색 결과를 Agent의 작업 노트와 같은 **State**에 저장하여 활용합니다.

❓ 오늘은 사람이 직접 알람 매뉴얼 문장을 찾아 프롬프트에 넣었습니다. 그렇다면 내일은 Agent가 직접 문서를 검색해서 근거를 찾게 만들면 어떨까요?

✅ ✨ 핵심 정리: 2일차는 Agent가 문서를 이해하고 활용하는 RAG 기능을 학습하는 과정입니다.

제조 장애 대응 Agent 기본 설계안

제조 장애 대응 Agent를 설계할 때는 “어떤 순서로 처리할지”를 먼저 정해야 합니다.



✳ 핵심 정리: Agent 설계의 핵심은 LLM 호출이 아니라, 입력 확인 → 조건 분기 → 근거 수집 → 응답 생성 흐름을 정하는 것입니다.